

Walkthrough del código fracciones_reparto_premio_v1.Rmd

Parte 1: Configuración inicial y metadatos

```
---
output:
  word_document: default
  pdf_document:
    keep_tex: true
    extra_dependencies: ["graphicx", "float", "tikz", "xcolor"]
  html_document: default
icfes:
  competencia: Resolución de problemas
  componente: Numérico-variacional
  afirmacion: Resuelve problemas que requieren el uso de fracciones y porcentajes
  evidencia: Utiliza fracciones para resolver problemas de reparto proporcional
  nivel: Medio
  tematica: Fracciones y operaciones con fracciones
---
```

Esta sección contiene:

- **Configuración de salida:** Define los formatos de salida (Word, PDF, HTML) y sus opciones.
- **Metadatos ICFES:** Información específica para categorizar el ejercicio según estándares ICFES (competencia, componente, afirmación, etc.).

Parte 2: Configuración de R y bibliotecas

```
```{r setup, include=FALSE}
Configuración para todos los formatos de salida
Sys.setlocale(category = "LC_NUMERIC", locale = "C")
options(OutDec = ".")

Configurar el motor LaTeX globalmente
options(tikzLatex = "pdflatex")
options(tikzXelatex = FALSE)
options(tikzLatexPackages = c(
 "\\usepackage{tikz}",
 "\\usepackage{colortbl}",
 "\\usepackage{xcolor}",
 "\\usepackage{graphicx}",
 "\\usepackage{float}"
))

library(exams)
library(reticulate)
library(digest)
library(testthat)
library(knitr)

typ <- match_exams_device()
options(scipen = 999)
knitr::opts_chunk$set(
 warning = FALSE,
 message = FALSE,
```

```

fig.showtext = FALSE,
fig.cap = "",
fig.keep = 'all',
dev = c("png", "pdf"),
dpi = 150,
fig.pos = "H"
)

Configuración para chunks de Python
knitr::knit_engines$set(python = function(options) {
 knitr::engine_output(options, options$code, '')
})

Asegurar que Python esté correctamente configurado
use_python(Sys.which("python"), required = TRUE)

```

Esta sección:

- **Configura el entorno R:** Establece la configuración regional para números (punto decimal).
- **Configura LaTeX:** Define opciones para el motor LaTeX.
- **Carga bibliotecas:** Importa las bibliotecas necesarias (exams, reticulate, testthat, etc.).
- **Configura knitr:** Establece opciones para los chunks de código.
- **Configura Python:** Asegura que Python esté disponible para usar con reticulate.

### Parte 3: Definición y aleatorización de variables

```

```{r DefinicionDeVariables, message=FALSE, warning=FALSE, results='asis'}
options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorización del contexto del problema
contextos <- c(
  "ciudad", "municipio", "localidad", "comunidad", "pueblo",
  "distrito", "barrio", "región", "provincia", "zona"
)
contexto <- sample(contextos, 1)

# Aleatorización del tipo de competencia
competencias <- c(
  "maratón", "carrera", "competencia atlética", "torneo deportivo",
  "olimpiada deportiva", "evento deportivo", "justa deportiva",
  "campeonato", "concurso deportivo", "prueba atlética"
)
competencia <- sample(competencias, 1)

# Aleatorización del grupo de edad
edades <- c(
  "menores de 15 años", "menores de 16 años", "menores de 14 años",
  "niños y niñas de 10 a 15 años", "jóvenes de 12 a 15 años",
  "estudiantes de primaria y secundaria", "categoría infantil",
  "categoría juvenil", "niños y adolescentes", "estudiantes menores de edad"
)

```

```

grupo_edad <- sample(edades, 1)

# Aleatorización del premio total (en millones)
# Generamos valores que sean divisibles por 10 para facilitar cálculos
premios_posibles <- c(30, 40, 50, 60, 70, 80, 90, 100, 120, 150)
premio_total <- sample(premios_posibles, 1)

```

Esta parte:

- **Establece una semilla aleatoria:** Garantiza reproducibilidad pero con variación entre ejecuciones.
- **Aleatoriza el contexto:** Selecciona aleatoriamente un contexto (ciudad, municipio, etc.).
- **Aleatoriza el tipo de competencia:** Selecciona un tipo de evento deportivo.
- **Aleatoriza el grupo de edad:** Selecciona una descripción para el grupo de participantes.
- **Aleatoriza el premio total:** Selecciona un valor entre 30 y 150 millones.

Parte 4: Aleatorización de términos y expresiones

```

# Aleatorización de términos para el enunciado
terminos_premiar <- c("premiar", "recompensar", "reconocer", "galardonar", "incentivar")
termino_premiar <- sample(terminos_premiar, 1)

terminos_participantes <- c("participantes", "competidores", "concursantes", "atletas", "deportistas")
termino_participantes <- sample(terminos_participantes, 1)

terminos_cuenta <- c("cuenta con", "dispone de", "tiene asignado", "ha destinado", "ha reservado")
termino_cuenta <- sample(terminos_cuenta, 1)

terminos_repartiran <- c("repartirán", "distribuirán", "dividirán", "asignarán", "otorgarán")
termino_repartiran <- sample(terminos_repartiran, 1)

terminos_puestos <- c("primeros puestos", "primeras posiciones", "ganadores", "mejores lugares", "primeros")
termino_puestos <- sample(terminos_puestos, 1)

terminos_dinero <- c("dinero", "premio", "recompensa", "incentivo", "monto")
termino_dinero <- sample(terminos_dinero, 1)

```

Esta sección:

- **Aleatoriza términos del enunciado:** Selecciona aleatoriamente diferentes verbos y sustantivos para variar la redacción del problema.
- Esto permite generar múltiples variantes del mismo problema con diferente redacción.

Parte 5: Aleatorización de fracciones y cálculos matemáticos

```

# Aleatorización de fracciones para los puestos
# Definimos conjuntos de fracciones que sumen menos de 1 para que quede algo para el tercer puesto
conjuntos_fracciones <- list(
  c("1/2", "2/5"), # Suma 9/10, queda 1/10
  c("1/3", "1/2"), # Suma 5/6, queda 1/6
  c("2/5", "1/2"), # Suma 9/10, queda 1/10
  c("3/5", "1/4"), # Suma 17/20, queda 3/20
  c("1/2", "1/3"), # Suma 5/6, queda 1/6
  c("3/4", "1/8"), # Suma 7/8, queda 1/8
  c("2/3", "1/5"), # Suma 13/15, queda 2/15

```

```

  c("3/5", "1/3"), # Suma 14/15, queda 1/15
  c("1/2", "3/8"), # Suma 7/8, queda 1/8
  c("3/5", "3/10") # Suma 9/10, queda 1/10
)

# Seleccionar un conjunto aleatorio de fracciones
indice_conjunto <- sample(1:length(conjuntos_fracciones), 1)
fracciones_seleccionadas <- conjuntos_fracciones[[indice_conjunto]]

# Asignar fracciones a los puestos
fraccion_primer_puesto <- fracciones_seleccionadas[1]
fraccion_segundo_puesto <- fracciones_seleccionadas[2]

# Convertir fracciones a valores numéricos para cálculos
convertir_fraccion <- function(fraccion) {
  partes <- strsplit(fraccion, "/")[[1]]
  return(as.numeric(partes[1]) / as.numeric(partes[2]))
}

valor_primer_puesto <- convertir_fraccion(fraccion_primer_puesto)
valor_segundo_puesto <- convertir_fraccion(fraccion_segundo_puesto)

# Calcular la fracción y el valor para el tercer puesto
valor_tercer_puesto <- 1 - (valor_primer_puesto + valor_segundo_puesto)

# Verificar que el valor del tercer puesto sea positivo
test_that("El valor del tercer puesto es positivo", {
  expect_true(valor_tercer_puesto > 0)
})

# Calcular los montos en millones para cada puesto
monto_primer_puesto <- premio_total * valor_primer_puesto
monto_segundo_puesto <- premio_total * valor_segundo_puesto
monto_tercer_puesto <- premio_total * valor_tercer_puesto

# Redondear a números enteros si es necesario
monto_primer_puesto <- round(monto_primer_puesto)
monto_segundo_puesto <- round(monto_segundo_puesto)
monto_tercer_puesto <- round(monto_tercer_puesto)

# Ajustar el tercer puesto para asegurar que la suma sea exactamente el premio total
suma_actual <- monto_primer_puesto + monto_segundo_puesto + monto_tercer_puesto
if (suma_actual != premio_total) {
  monto_tercer_puesto <- premio_total - (monto_primer_puesto + monto_segundo_puesto)
}

# Verificar que la suma de los tres montos sea igual al premio total
test_that("La suma de los tres montos es igual al premio total", {
  expect_equal(monto_primer_puesto + monto_segundo_puesto + monto_tercer_puesto, premio_total)
})

```

Esta sección:

- **Define conjuntos de fracciones:** Crea 10 conjuntos diferentes de fracciones para el primer y segundo

puesto.

- **Selecciona un conjunto aleatorio:** Elige uno de los conjuntos para usar en el problema.
- **Convierte fracciones a valores numéricos:** Crea una función para convertir fracciones en texto a valores decimales.
- **Calcula el valor del tercer puesto:** Resta del total (1) la suma de los valores del primer y segundo puesto.
- **Verifica la validez:** Comprueba que el valor del tercer puesto sea positivo.
- **Calcula los montos en millones:** Multiplica los valores por el premio total.
- **Ajusta para coherencia:** Asegura que la suma de los tres montos sea exactamente igual al premio total.

Parte 6: Generación de opciones de respuesta

```
# Generar opciones de respuesta
# La respuesta correcta es el monto del tercer puesto
respuesta_correcta <- monto_tercer_puesto

# Generar distractores plausibles
distractor1 <- round(premio_total * 0.05) # 5% del premio total
distractor2 <- round(premio_total * 0.2)  # 20% del premio total
distractor3 <- round(premio_total * 0.3)  # 30% del premio total

# Asegurarse de que todos los distractores son diferentes de la respuesta correcta
if (distractor1 == respuesta_correcta) distractor1 <- distractor1 + 1
if (distractor2 == respuesta_correcta) distractor2 <- distractor2 + 2
if (distractor3 == respuesta_correcta) distractor3 <- distractor3 - 2

# Asegurarse de que todos los distractores son diferentes entre sí
while (length(unique(c(distractor1, distractor2, distractor3))) < 3) {
  if (distractor1 == distractor2) distractor1 <- distractor1 + 1
  if (distractor2 == distractor3) distractor2 <- distractor2 + 2
  if (distractor1 == distractor3) distractor3 <- distractor3 + 3
}

# Crear un vector con todas las opciones y mezclarlas
opciones <- c(respuesta_correcta, distractor1, distractor2, distractor3)
names(opciones) <- c("correcta", "distractor1", "distractor2", "distractor3")
opciones_mezcladas <- sample(opciones)

# Identificar la posición de la respuesta correcta en las opciones mezcladas
indice_correcto <- which(opciones_mezcladas == respuesta_correcta)

# Crear el vector de solución para r-exams
solucion <- rep(0, 4)
solucion[indice_correcto] <- 1
```

Esta sección:

- **Define la respuesta correcta:** El monto calculado para el tercer puesto.
- **Genera distractores plausibles:** Crea tres opciones incorrectas pero plausibles.
- **Asegura la unicidad:** Verifica que todas las opciones sean diferentes entre sí.
- **Aleatoriza el orden:** Mezcla las opciones para que la respuesta correcta no siempre esté en la misma posición.
- **Crea el vector de solución:** Genera un vector binario donde 1 indica la respuesta correcta.

Parte 7: Aleatorización de colores y generación de la tabla

```
# Aleatorización de colores para la tabla
paletas_colores <- list(
  c("#4285F4", "#EA4335", "#FBBC05", "#34A853"), # Google colors
  c("#1F77B4", "#FF7F0E", "#2CA02C", "#D62728"), # Tableau colors
  c("#003f5c", "#58508d", "#bc5090", "#ff6361"), # Viridis-like
  c("#0073C2", "#EFC000", "#868686", "#CD534C"), # IBM colors
  c("#7F3C8D", "#11A579", "#3969AC", "#F2B701") # Colorbrewer
)
paleta_seleccionada <- sample(paletas_colores, 1)[[1]]
color_encabezado <- paleta_seleccionada[1]
color_primer_puesto <- paleta_seleccionada[2]
color_segundo_puesto <- paleta_seleccionada[3]
color_tercer_puesto <- paleta_seleccionada[4]
```

Esta sección:

- **Define paletas de colores:** Crea 5 paletas diferentes con combinaciones armónicas.
- **Selecciona una paleta aleatoria:** Elige una de las paletas para usar en la visualización.
- **Asigna colores:** Distribuye los colores para el encabezado y cada puesto.

Parte 8: Generación de la tabla con Python/Matplotlib

```
```{r generar_tabla_tikz, results='asis'}
Crear código Python para generar la tabla usando matplotlib
codigo_python <- paste0('
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.gridspec import GridSpec
import numpy as np

Configurar colores
color_encabezado = '', color_encabezado, ''
color_primer_puesto = '', color_primer_puesto, ''
color_segundo_puesto = '', color_segundo_puesto, ''
color_tercer_puesto = '', color_tercer_puesto, ''

Crear figura y configurar grid
fig = plt.figure(figsize=(8, 3.5))
gs = GridSpec(4, 2, height_ratios=[1, 1, 1, 1], width_ratios=[3, 7])

Encabezado (ocupa todo el ancho)
ax_header = plt.subplot(gs[0, :])
ax_header.set_facecolor(color_encabezado)
ax_header.text(0.5, 0.5, "Distribución del premio de ", premio_total, ' millones de pesos",
 ha="center", va="center", color="white", fontweight="bold", fontsize=11)
ax_header.set_xticks([])
ax_header.set_yticks([])
ax_header.spines["top"].set_visible(True)
ax_header.spines["bottom"].set_visible(True)
ax_header.spines["left"].set_visible(True)

```

```

ax_header.spines["right"].set_visible(True)

Primera fila
ax_p1_label = plt.subplot(gs[1, 0])
ax_p1_label.set_facecolor(color_primer_puesto)
ax_p1_label.text(0.5, 0.5, "Primer puesto", ha="center", va="center", color="white", fontweight="bold",
ax_p1_label.set_xticks([])
ax_p1_label.set_yticks([])
ax_p1_label.spines["top"].set_visible(True)
ax_p1_label.spines["bottom"].set_visible(True)
ax_p1_label.spines["left"].set_visible(True)
ax_p1_label.spines["right"].set_visible(True)

... [código similar para las demás celdas de la tabla]

Guardar la figura
plt.savefig("tabla_distribucion.png", dpi=150, bbox_inches="tight")
plt.savefig("tabla_distribucion.pdf", dpi=150, bbox_inches="tight")
plt.close()
')

Ejecutar código Python para generar la figura
py_run_string(codigo_python)

```

Esta sección:

- **Genera código Python:** Crea dinámicamente código Python con los valores y colores aleatorizados.
- **Crea una tabla visual:** Utiliza Matplotlib para generar una tabla con el mismo aspecto que tendría en TikZ.
- **Guarda la figura:** Almacena la tabla en formatos PNG y PDF para su uso en diferentes salidas.

## Parte 9: Formulación de la pregunta

Question

=====

En un(a) ``r contexto`` se realizará un(a) ``r competencia`` para ``r grupo_edad``. Para ``r termino_premiar`` a

```

```{r tabla_distribucion, echo=FALSE, results='asis', fig.align='center'}
# Detectar si se está generando para Moodle u otros formatos
formatos_moodle <- c("exams2moodle", "exams2qti12", "exams2qti21", "exams2openolat")
es_moodle <- (match_exams_call() %in% formatos_moodle)

# Incluir la imagen generada por Python
if (es_moodle) {
  # Tamaño para Moodle
  cat("![[](tabla_distribucion.png){width=80%}")
} else {
  # Tamaño para PDF/Word
  cat("![[](tabla_distribucion.png){width=90%}")
}

```

¿Qué cantidad de `r termino_dinero` recibe el tercer puesto?

Answerlist

- r opciones_mezcladas[1] millones.
- r opciones_mezcladas[2] millones.
- r opciones_mezcladas[3] millones.
- r opciones_mezcladas[4] millones.

Esta sección:

- **Formula la pregunta**: Utiliza las variables aleatorizadas para crear el enunciado del problema.
- **Incluye la tabla**: Inserta la tabla generada con Python, adaptando su tamaño según el formato de salida.
- **Presenta las opciones**: Muestra las opciones de respuesta mezcladas.

Parte 10: Solución detallada

```
```r
Solution
=====
```

Para resolver este problema, debemos calcular qué fracción del premio total corresponde al tercer puesto.

### Paso 1: Identificar los datos del problema

- Premio total: `r premio\_total` millones de pesos
- Primer puesto: `r fraccion\_primer\_puesto` del dinero total
- Segundo puesto: `r fraccion\_segundo\_puesto` del dinero total
- Tercer puesto: el dinero restante

### Paso 2: Convertir las fracciones a decimales

- Primer puesto: `r fraccion\_primer\_puesto` = `r valor\_primer\_puesto`
- Segundo puesto: `r fraccion\_segundo\_puesto` = `r valor\_segundo\_puesto`

### Paso 3: Calcular la fracción que corresponde al tercer puesto

Para calcular la fracción del tercer puesto, restamos del total (1) las fracciones del primer y segundo puesto:

- Fracción del tercer puesto =  $1 - (\text{r valor\_primer\_puesto} + \text{r valor\_segundo\_puesto})$
- Fracción del tercer puesto =  $1 - \text{r valor\_primer\_puesto} + \text{valor\_segundo\_puesto}$
- Fracción del tercer puesto = `r valor\_tercer\_puesto`

### Paso 4: Calcular el monto en millones de pesos para el tercer puesto

Multiplicamos la fracción del tercer puesto por el premio total:

- Monto del tercer puesto = `r valor\_tercer\_puesto` × `r premio\_total` millones
- Monto del tercer puesto = `r monto\_tercer\_puesto` millones de pesos

### Verificación

Comprobemos que la suma de los tres montos es igual al premio total:

- Primer puesto: `r monto\_primer\_puesto` millones
- Segundo puesto: `r monto\_segundo\_puesto` millones
- Tercer puesto: `r monto\_tercer\_puesto` millones
- Total: `r monto\_primer\_puesto + monto\_segundo\_puesto + monto\_tercer\_puesto` millones

Como `r monto\_primer\_puesto + monto\_segundo\_puesto + monto\_tercer\_puesto` = `r premio\_total`, confirmamos que la suma es correcta.

Por lo tanto, el tercer puesto recibe `r monto\_tercer\_puesto` millones de pesos.

Answerlist



```

- `r if(solucion[1] == 1) "Verdadero" else "Falso"`
- `r if(solucion[2] == 1) "Verdadero" else "Falso"`
- `r if(solucion[3] == 1) "Verdadero" else "Falso"`
- `r if(solucion[4] == 1) "Verdadero" else "Falso"`
```

Esta sección:

- **Explica la solución paso a paso:** Detalla el proceso de resolución del problema.
- **Muestra los cálculos:** Incluye los valores numéricos en cada paso.
- **Verifica la respuesta:** Comprueba que la suma de los tres montos sea igual al premio total.
- **Indica la respuesta correcta:** Marca cuál de las opciones es la correcta.

## Parte 11: Metainformación para r-exams

```
Meta-information
=====
exname: fracciones_reparto_premio
extype: schoice
exsolution: `r paste(as.integer(solucion), collapse="")`
exshuffle: TRUE
exsection: Aritmética|Fracciones|Reparto proporcional
```

Esta sección:

- **Define el nombre del ejercicio:** Asigna un identificador único.
- **Especifica el tipo de ejercicio:** Indica que es una pregunta de selección única (schoice).
- **Codifica la solución:** Convierte el vector de solución en una cadena de 0s y 1s.
- **Habilita la aleatorización de opciones:** Permite que las opciones se mezclen en cada generación.
- **Categoriza el ejercicio:** Asigna etiquetas temáticas para facilitar la organización.